

Project Report: Health Statistics Dashboard

An Interactive Web App for Analyzing Patient and Clinical Data

NAME: Awais Javid

Date: June 21, 2026

Software: RStudio & Shiny App

Platform: shinyapps.io (Posit)

1. Introduction

It is important to be able to view patient information rapidly, especially in the health care and public health fields, where they often need to spot health trends. But it can be time-consuming and difficult to interpret the raw tables or spreadsheets.

This project explains how to create a web application that is user-friendly and visual for data analysis, the Health Statistics Dashboard. This app was developed in R programming language and Shiny, and the primary purpose was to enable users to easily filter health information. They can access up to the minute updates in charts and tables without having to write any code or database queries.

2. Why I Built This Project

Most public health reports are static, that is, they are available to download as a pdf or as a static image that cannot be modified. It's likely that if a doctor needs to access data for a particular age group or for a specific disease, he will need to request it from a data analyst to produce a new report. This dashboard does that by allowing you to:

- **Live Filters:** Users can adjust settings like age or gender and see the charts update immediately.
- **Clear Summaries:** The app automatically calculates key averages, such as BMI and blood pressure, for the selected group of patients.
- **Simple Visuals:** Complex medical data is transformed into easy-to-read charts that highlight health patterns clearly.

3. The Dataset

For testing purposes, I created a realistic sample data set with 500 records of patients. Each health marker has specific ranges to make the data reflect a real-world clinic population:

- **Basic Information:** Patients are aged between 18 and 80 years old, with an equal split between male and female entries.
- **Health Conditions:** Patients are randomly assigned to one of four categories: Diabetes, Hypertension, Heart Disease, or Healthy.
- **Medical Metrics:** I simulated standard health indicators, including Body Mass Index (BMI), blood pressure, and cholesterol levels, ensuring they align with typical biological averages.

Note on Code Consistency: I included the `set.seed(123)` command in the code. This ensures that the app generates the exact same random numbers every time it runs, making the project easy to test and verify.

4. How the App Works (Architecture)

The application is split into two main parts that communicate with each other directly: the User Interface (what the user sees) and the Server logic (the brain of the app).

4.1 The User Interface (UI)

I used the shiny dashboard package to design a clean, organized layout. On the left side, a sidebar contains control widgets, including an age slider and dropdown menus for selecting gender and diseases. The main section of the screen displays the results using interactive boxes, charts, and a large data table.

4.2 Server Logic and Reactivity

The server side relies on a smart feature called a "reactive expression" (`reactive()`). When a user changes a filter in the sidebar, the app does not reload the entire dataset multiple times for each chart. Instead, it filters the data once in the background, holds it in the memory, and updates all the charts simultaneously. This keeps the application fast and responsive.

Dashboard Core Features

Dashboard Feature	R Package Used	What It Shows to the User
Value Boxes	shiny dashboard	Displays the total patient count, average BMI, and average blood pressure for the filtered group.

Dashboard Feature	R Package Used	What It Shows to the User
Disease & Gender Charts	ggplot2	A bar chart showing the number of cases for each illness and a pie chart for the gender distribution.
Age Histogram	ggplot2	A visual breakdown showing how patients are distributed across different age groups.
Data Table	DT	A searchable spreadsheet list where users can sort data and view raw patient metrics.

5. Deployment and Version Control

To ensure this project is accessible to others and ready for a professional portfolio, I moved the code from my local computer and deployed it to the web using two main tools:

- **Live Website Deployment:** I consolidated the code into a single file named app.R and uploaded it to **shinyapps.io**. This platform automatically handles all 60 required package configurations, creating a working web link that anyone can open.
- **GitHub Repository:** I set up a public repository on GitHub to host the source code. This allows others to review my clean code and easily follow the project documentation.

6. Conclusion and Future Ideas

This dashboard is a great example of how easy it is to use R and Shiny to transform complex numbers into something easy-to-use and user-friendly. It provides healthcare professionals with the ability to now explore the patients' groups as if they were in the room without requiring data science skills.

This application could be integrated with existing EHR systems in the future to present current clinical information. It could also be enhanced with simple machine learning algorithms to facilitate prediction of health risks from patient data.